Cloud Native Data Engineering — Assignment 2

Design & Deploy: Relational Modeling, SQL & Cloud Database Implementation

Campus Energy Monitoring Platform

Obaro Erigha

Autumn 2026 · Amazon RDS for MySQL 8.4 · us-east-2

Contents

Overview — How Assignment 2 Evolves Assignment 1 3
Part 1 — Dataset Acquisition & Profiling 4
1.1 Acquisition option: Option B (public real-world data) 4
1.2 Construction 4
1.3 Profile 4
Part 2 — Normalization (1NF → 2NF → 3NF) 6
2.1 Insertion anomaly 6
2.2 Deletion anomaly 6
2.3 Update anomaly 6
2.4 Functional dependencies (verified against all 28,336 rows) 6
2.5 Decomposition and business assumptions 7
Part 3 — Conceptual and Logical Data Model 8
3a. Conceptual ER diagram 8
3b. Logical ER diagram 10
3c. Structural requirements 11
Part 4 — Deploy to AWS RDS 12
4a. Provisioning 12
4b. Schema creation and data loading 12
4c. Verification evidence 13
Part 5 — Business Queries 14
Q1. INNER JOIN x6, GROUP BY, HAVING 14
Q2. LEFT OUTER JOIN, GROUP BY, HAVING 15
Q3. CTE + window function LAG 15
Q4. CTE + window function ROW_NUMBER, multi-table INNER JOIN 16
Q5. Recursive hierarchy (self-join on space_usage), GROUP BY WITH ROLLUP 17
Q6. INNER JOIN incl. weather, CASE banding, GROUP BY, HAVING 18
Q7. CTE + window functions AVG/STDDEV OVER (PARTITION BY) 19
Q8. RIGHT OUTER JOIN + LEFT OUTER JOIN, GROUP BY 20
Part 6 — Optimization, With Proof 22
6a. Execution plans — two most join-heavy queries 22
6b. Index hypothesis and before/after 22
6c. Empirical analysis 23
Part 7 — Cost, Teardown & Cloud Hygiene 24
7a. Billing 24
7b. Teardown 24
7c. Production 24/7 cost estimate 25
Appendix A — Full EXPLAIN ANALYZE Output (RDS MySQL 8.4.11) 26
Appendix B — Notes, Reproducibility and Deliverables 28

Overview — How Assignment 2 Evolves Assignment 1

Assignment 1 designed a Campus Energy Monitoring Platform for a university facilities team running 124 buildings and 308 hourly meters at the BDG2 "Bull" site (Austin, TX), combining building metadata, meter readings, weather and EIA electricity prices. Assignment 2 implements that platform's relational serving layer: the same buildings, meters, weather and price sources are normalized to 3NF, deployed to Amazon RDS, and queried for the A1 success criteria (cost attribution per building, data completeness ≥ 95%, anomaly detection, peak demand).

Decision	Assignment 1	Assignment 2	Why
Relational engine	RDS PostgreSQL	RDS MySQL 8.4.11	A2 spec requires RDS MySQL; 8.4 chosen over 8.0 because 8.0 entered paid RDS Extended Support on 2026-08-01 (verified via AWS API).
Data grain	Hourly readings	Daily per meter (Jul–Sep 2017)	28,336 rows fits the 5k–50k free-tier guidance and covers Austin's summer cooling peak.
Data source	BDG2 + EIA	BDG2 + EIA-861M (real data only)	Same sources; EIA-861M gives real Texas commercial monthly prices.
Scope	Full lake + warehouse	Relational model on RDS	A2 focuses on the OLTP/serving tier; the lake tiers remain A1 design.

Part 1 — Dataset Acquisition & Profiling

1.1 Acquisition option: Option B (public real-world data)

No synthetic records were generated. Two public sources were reshaped by a documented, reproducible script (scripts/build_flat_table.py) into one flat, denormalized table.

Source	Provider	URL	License	Downloaded
Building Data Genome Project 2 (commit 9b97ccbe)	BUDS Lab, NUS (Miller et al., 2020)	github.com/buds-lab/building-data-genome-project-2	CC BY-SA 4.0	2026-10-31
EIA-861M Monthly Sales & Revenue (sales_revenue.xlsx)	U.S. Energy Information Administration	eia.gov/electricity/data/eia861m/xls/sales_revenue.xlsx	U.S. Gov. public domain	2026-11-01 (sha256 c7963ad7…)

Citation: Miller, C. et al. The Building Data Genome Project 2, energy meter data from the ASHRAE Great Energy Predictor III competition. Sci Data 7, 368 (2020). doi:10.1038/s41597-020-00712-x. Per the BDG2 paper (Tables 4–5), all energy meters are reported in kWh (chilled water converted from tons × 3.51685).

1.2 Construction

- Filtered BDG2 metadata to site "Bull": 124 buildings.
- Read cleaned hourly meters for electricity (123), chilled water (95) and steam (90) = 308 meters; restricted to 2017-07-01 00:00 – 2017-09-30 23:00; unpivoted wide → long; aggregated per meter per day (sum, max hour, hour of max, hours reported).
- Aggregated Bull hourly weather to daily values; joined the EIA-861M Texas commercial average retail price for Jul/Aug/Sep 2017 (8.24 / 8.17 / 8.16 ¢/kWh, status Final).
- Applied unit rates: electricity = EIA price; chilled water = EIA price ÷ chiller COP 4.0 (assumption); steam = $0.034/kWh (gas boiler assumption).

1.3 Profile

Rows: 28,336 (308 meters × 92 days; 0 duplicate meter-days) · **Columns: 29** · Rows by utility: electricity 11,316, chilled water 8,740, steam 8,280.

Column(s)	Type	Nulls	Business concept
reading_id, reading_date	int, date	0	Reading
meter_code ("building|utility")	text	0	Meter (non-atomic composite)
utility_type, unit	text	0	Utility
building_name, building_kaggle_id, gross_sqm, gross_sqft	text, int, decimal	0	Building
primary_space_usage, sub_space_usage	text	0	Space-usage category (parent / child)
site_name, timezone, lat, lng	text, decimal	0	Site
daily_consumption_kwh, peak_hour_kwh, peak_hour	decimal, int	237 each	Reading measures
hours_reported	int 0–24	0	Data quality
avg/max_air_temp_c, avg_dew_temp_c, total_precip_mm, weather_hours	decimal, int	0	Weather (site-day)
tx_comm_price_cents_kwh, eia_data_status, unit_rate_usd, rate_source	decimal, text	0	Tariff (utility × month)
est_daily_cost_usd	decimal	237	Derived cost

Five-row preview (selected columns)

reading_id	reading_date	meter_code	primary / sub usage	gross_sqm	daily kWh	peak kWh @hr	hrs	rate $	cost $
1	2017-07-01	Bull_assembly_Amalia|chilledwater	Entertainment / Sports Stadium	52,377.0	98,524.7	4,450.8 @15	24	0.0206	2,029.61

reading_id	reading_date	meter_code	primary / sub usage	gross_sqm	daily kWh	peak kWh @hr	hrs	rate $	cost $
2	2017-07-01	Bull_assembly_Amalia|electricity	Entertainment / Sports Stadium	52,377.0	9,366.0	415.0 @22	24	0.0824	771.76
3	2017-07-01	Bull_assembly_Amalia|steam	Entertainment / Sports Stadium	52,377.0	1,916.3	100.5 @7	24	0.0340	65.15
4	2017-07-01	Bull_assembly_Beau|chilledwater	Entertainment / Conference Center	46,789.1	72,685.3	3,421.2 @1	24	0.0206	1,497.32
5	2017-07-01	Bull_assembly_Beau|electricity	Entertainment / Conference Center	46,789.1	9,774.0	523.0 @0	24	0.0824	805.38

Real anomalies present (input to Part 2)

- Repeated entity data: building name, usage and area repeat on every meter-day (e.g., Bull_lodging_Caren on 276 rows); site/timezone/lat/lng on all 28,336 rows. These play the role of the spec's "repeated customer names and addresses".
- Inconsistent casing: sub-usage 'Office' (16 buildings) vs 'office' (1), and lower-case 'garage' (5).
- Nulls: 237 meter-days with no readings (Bull_office_Mai electricity is null for all 92 days); 698 partial days.
- Non-atomic value: meter_code packs building and utility into one string.

Part 2 — Normalization (1NF → 2NF → 3NF)

The step-by-step decomposition is in the workbook normalization_workbook.xlsx (tabs 1NF, 2NF, 3NF). All rows below are actual rows from the flat table.

2.1 Insertion anomaly

Bull_office_Mai's electricity meter reported nothing in the window, yet the only way to record that the meter exists is as reading rows — so it occupies 92 rows whose measures are all NULL. Conversely, a new building, a new usage category, or the October-2017 EIA price cannot be recorded until some meter reports a reading.

reading_id	reading_date	meter_code	sub_space_usage	gross_sqm	daily_consumption_kwh	hours_reported
280	2017-07-01	Bull_office_Mai|electricity	office	792.0	NULL	0

2.2 Deletion anomaly

Bull_assembly_Maren is the only Swimming Pool building. Purging its 276 readings (e.g., meters decommissioned) would also erase its floor area (11,345.5 m²) and the fact that 'Swimming Pool' is a sub-category of 'Entertainment/public assembly'.

reading_id	reading_date	meter_code	primary_space_usage	sub_space_usage	gross_sqm	kWh
34	2017-07-01	Bull_assembly_Maren|chilledwater	Entertainment/public assembly	Swimming Pool	11345.5	35450.5750
35	2017-07-01	Bull_assembly_Maren|electricity	Entertainment/public assembly	Swimming Pool	11345.5	7721.0000
36	2017-07-01	Bull_assembly_Maren|steam	Entertainment/public assembly	Swimming Pool	11345.5	302.3312

2.3 Update anomaly

Bull_office_Mai is stored as 'office' while 16 other buildings use 'Office'. Correcting it requires updating all 276 of its rows; missing any leaves one building with two usages and splits GROUP BY results. The same applies to any gross_sqm correction (276 rows) or an EIA price revision (August price repeated on 9,548 rows).

reading_id	reading_date	meter_code	primary_space_usage	sub_space_usage
279	2017-07-01	Bull_office_Mai|chilledwater	Office	office
280	2017-07-01	Bull_office_Mai|electricity	Office	office
281	2017-07-01	Bull_office_Mai|steam	Office	office
587	2017-07-02	Bull_office_Mai|chilledwater	Office	office

2.4 Functional dependencies (verified against all 28,336 rows)

FD	Dependency	Type
FD1	{building_name, utility_type, reading_date} → consumption, peak_hour_kwh, peak_hour, hours_reported	full (1NF key)
FD2	building_name → building_kaggle_id, site_name, primary/sub usage, gross_sqm, gross_sqft	partial
FD3	building_kaggle_id → building_name	alternate key
FD4	utility_type → unit	partial
FD5	site_name → timezone, lat, lng	transitive
FD6	sub_space_usage → primary_space_usage	transitive
FD7	gross_sqm → gross_sqft (× 10.7639)	derived
FD8	{site_name, reading_date} → weather measures	partial
FD9	reading_date → reading_month → TX price, EIA status	partial + transitive
FD10	{utility_type, reading_month} → unit_rate_usd, rate_source	partial

FD	Dependency	Type
FD11	{consumption, unit_rate_usd} → est_daily_cost_usd	derived

2.5 Decomposition and business assumptions

- **1NF:** split non-atomic meter_code; drop the row-number surrogate; key {building_name, utility_type, reading_date}.
- **2NF:** remove partial dependencies (FD2, FD4, FD8, FD9, FD10) → R_building (124), R_utility (3), R_day_weather (92), R_utility_day_rate (276), R_reading (28,336).
- **3NF:** remove transitive/derived dependencies (FD5, FD6, FD7, FD9, FD11) → 9 tables, 28,897 rows. A lossless-join check re-joins them to all 28,336 flat rows with identical values.

Business assumptions:

- A building has at most one meter per utility, so (building, utility) identifies a meter (308 pairs = 308 meters).
- Each sub-usage belongs to exactly one primary usage (0 violations).
- Weather is observed per site per day, not per building.
- Tariffs are set per utility per calendar month; the stored unit rate is a business (contract) decision.
- Sub-usage casing is standardized on load ('office' → 'Office', 'garage' → 'Garage'); the raw anomaly remains visible in the 1NF/2NF tabs.
- Derived values (gross_sqft, cost) are not stored; they are computed in SQL.

Part 3 — Conceptual and Logical Data Model

3a. Conceptual ER diagram

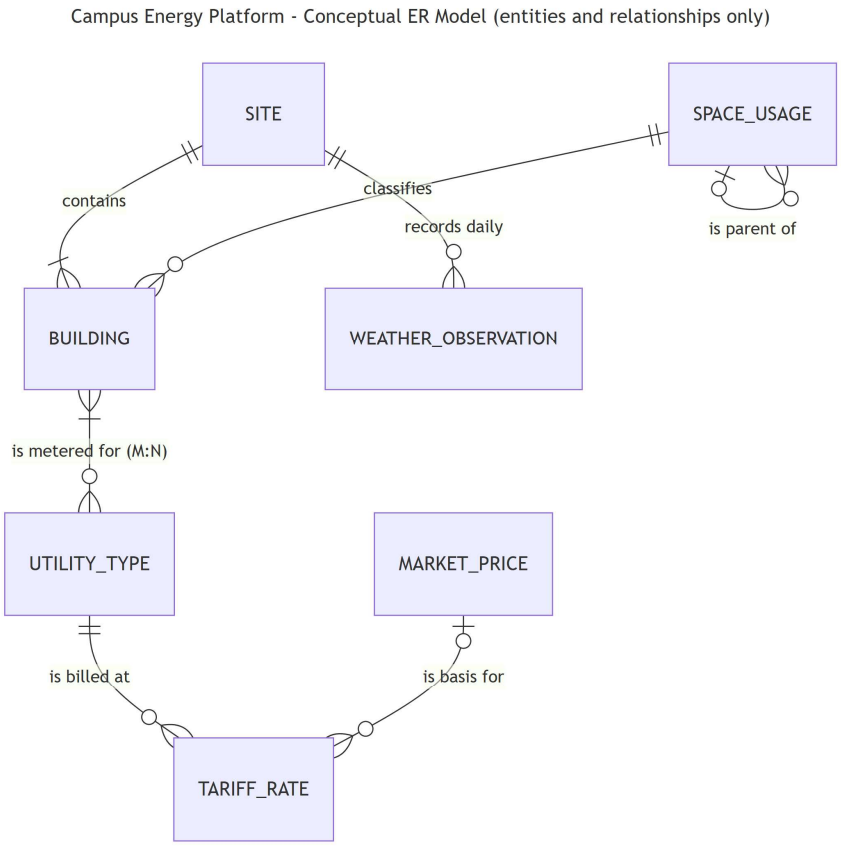


Figure 3.1 — Conceptual model (entities and relationships only); BUILDING–UTILITY_TYPE shown as M:N.

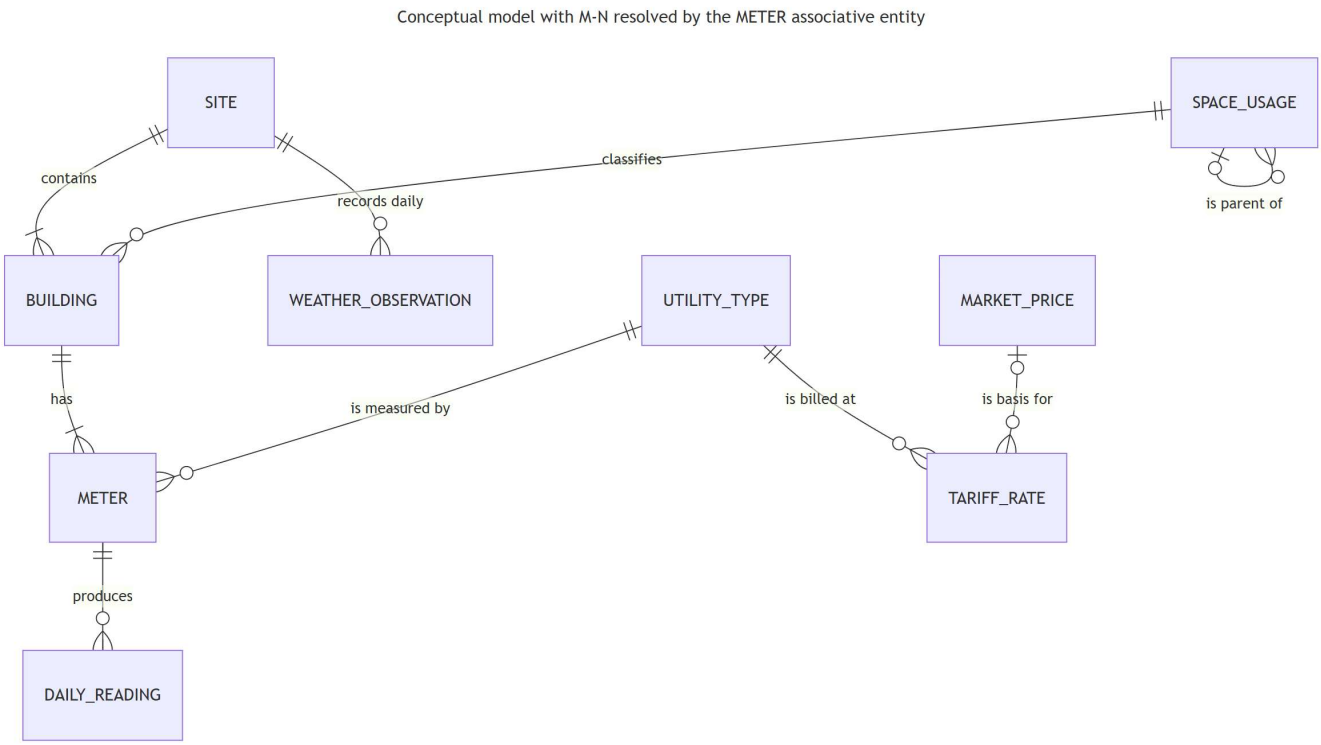


Figure 3.2 — Conceptual model with the M:N resolved by the METER associative entity.

3b. Logical ER diagram

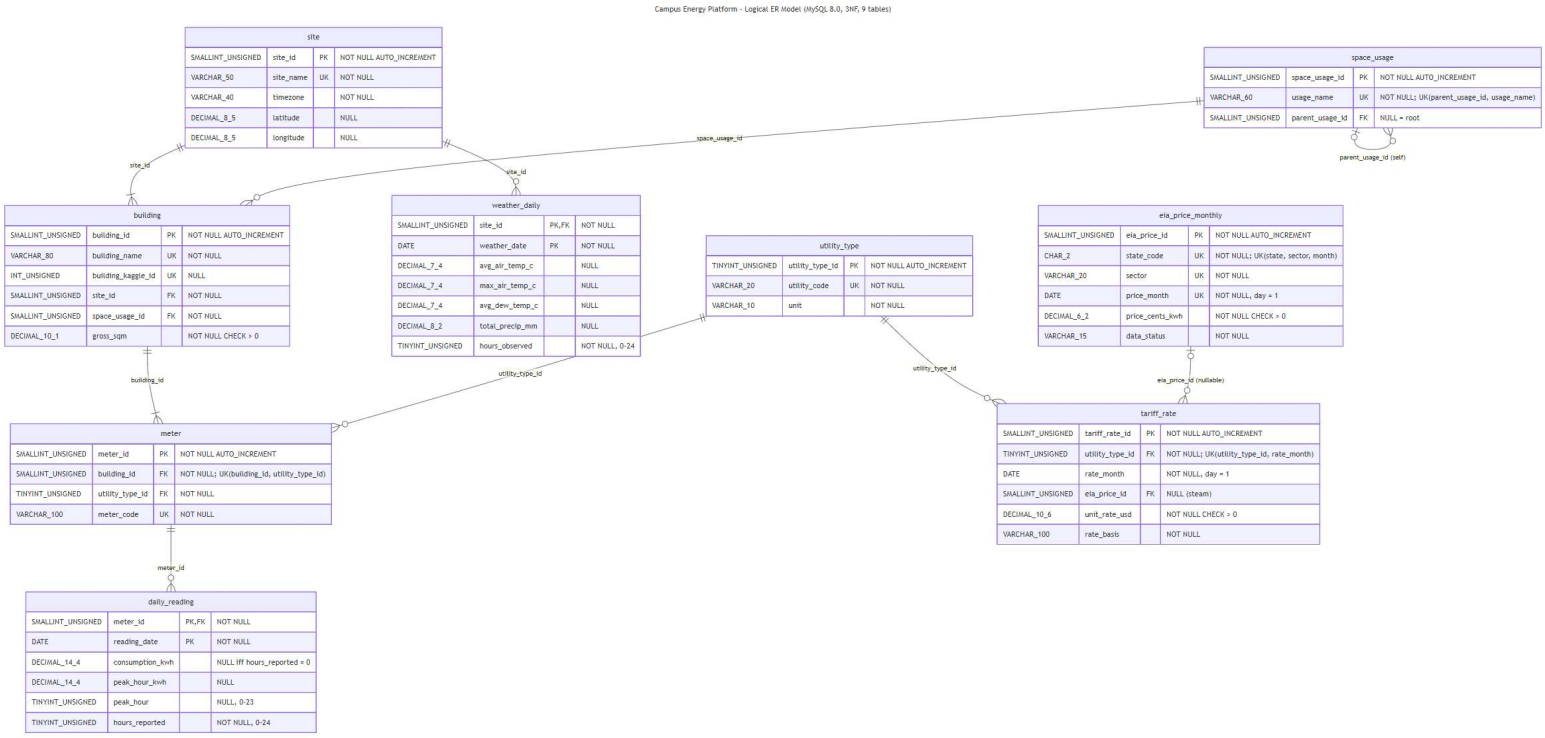


Figure 3.3 — Logical model: 9 tables, MySQL types (DECIMAL_10_1 = DECIMAL(10,1)), PK/FK/UK, NULL rules, cardinalities.

3c. Structural requirements

Requirement	Met by	Evidence
≥ 6 entities	9 tables: site, space_usage, building, utility_type, meter, daily_reading, weather_daily, eia_price_monthly, tariff_rate	SHOW TABLES on RDS (Screenshot 4.2)
M:N via junction	building ↔ utility_type resolved by meter (UNIQUE building_id, utility_type_id)	308 meters; buildings with 3/2/1 utilities: 88/8/28
Recursive hierarchy	space_usage.parent_usage_id → space_usage (e.g., Education → College Laboratory)	WITH RECURSIVE walk; Q5 rollup

Design choices (vs. the rejected alternative)

- **Meter as associative entity** rather than a bare building_utility bridge: the physical meter is the real asset that produces readings.
- **Self-referencing hierarchy** rather than separate primary/sub tables: depth can grow (e.g., Laboratory → Wet Lab) without a schema change.
- **Composite PK (meter_id, reading_date)** enforces one reading per meter per day and clusters each meter's time series.
- **ON DELETE RESTRICT** removes the Part 2 deletion anomaly (a used category cannot vanish); tariff → EIA price uses SET NULL.
- **DECIMAL** for energy and money: the SQL cost total ($14,832,619.22) matches an exact Decimal recomputation to the cent.

Part 4 — Deploy to AWS RDS

4a. Provisioning

Setting	Value
Identifier / engine	a2-campus-energy · MySQL 8.4.11 (Extended Support explicitly disabled)
Class / storage	db.t3.micro (2 vCPU, 1 GiB) · 20 GiB gp2 · storage autoscaling OFF
Availability / backups	Single-AZ (us-east-2a) · automated backup retention 0
Monitoring	Performance Insights off · Enhanced Monitoring off
Security	Encrypted at rest (aws/rds KMS key) · TLS client connections verified against the RDS CA bundle · publicly accessible but security group allows TCP 3306 from one /32 (the author's IP) only
Credentials	Master password entered in a hidden prompt; never written to disk or shared

Commands (region passed explicitly; IDs abbreviated):

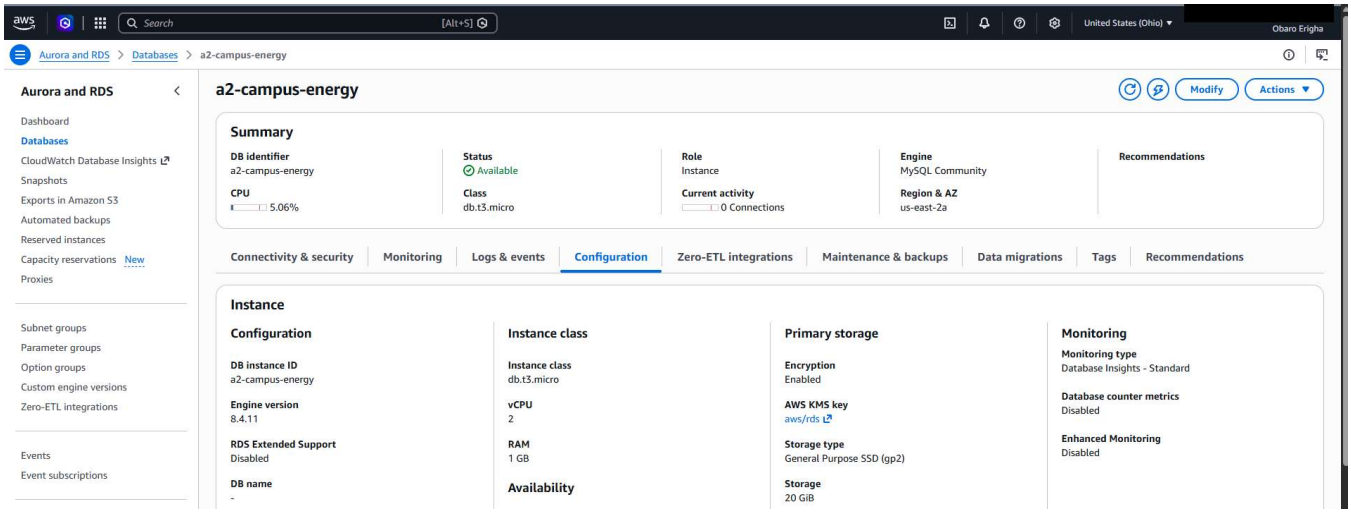
```
aws ec2 create-security-group --group-name a2-rds-mysql-sg --vpc-id <default-vpc> --region us-east-2 ...
aws ec2 authorize-security-group-ingress --group-id <sg> --protocol tcp --port 3306 --cidr <my-ip>/32 --region us-east-2
aws rds create-db-instance --db-instance-identifier a2-campus-energy --engine mysql --engine-version 8.4.11 \
   --engine-lifecycle-support open-source-rds-extended-support-disabled --db-instance-class db.t3.micro \
   --allocated-storage 20 --storage-type gp2 --no-multi-az --publicly-accessible --vpc-security-group-ids <sg> \
   --master-username a2admin --master-user-password <hidden> --backup-retention-period 0 \
   --no-enable-performance-insights --monitoring-interval 0 --storage-encrypted --region us-east-2
```

4b. Schema creation and data loading

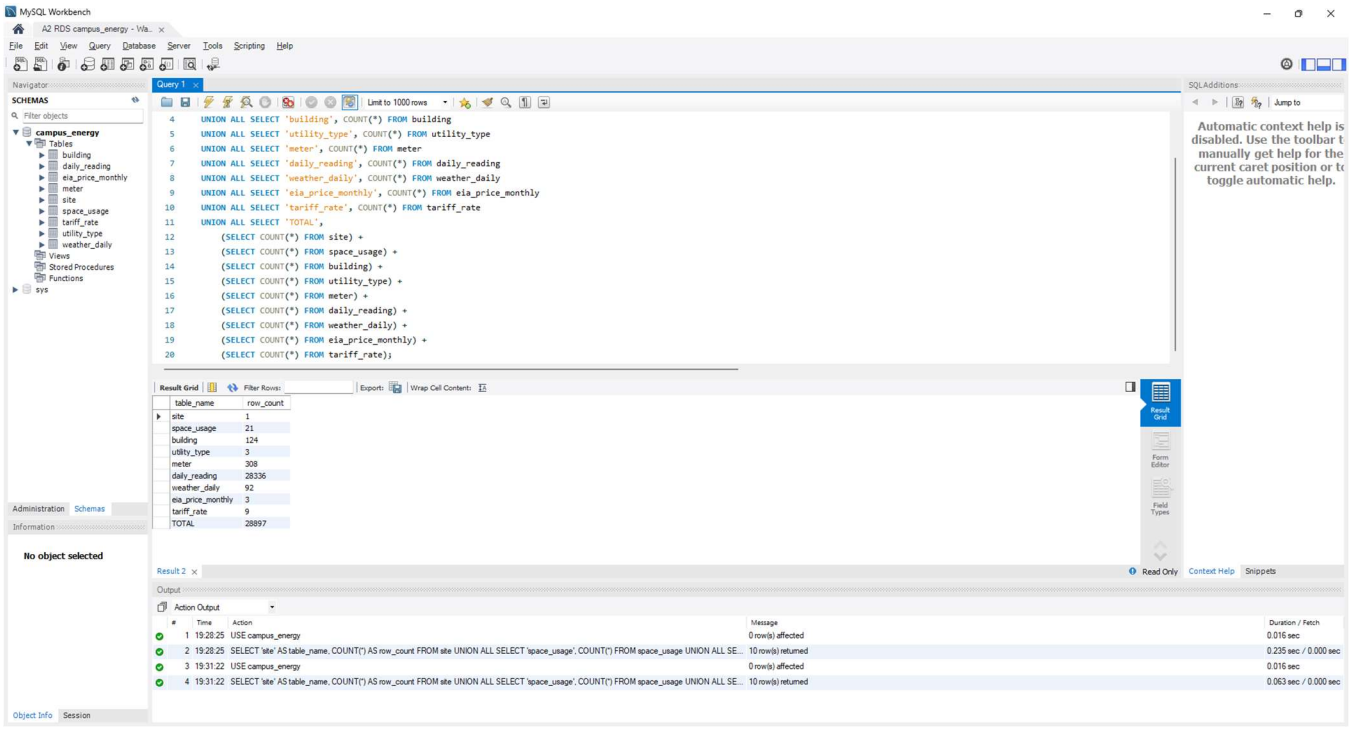
schema.sql (9 tables, 9 foreign keys, 12 CHECK constraints) was executed on RDS over TLS, then scripts/load_3nf.py (pymysql) loaded the 9 normalized CSVs in foreign-key order. The schema and loader were first validated on a local MySQL instance (re-runnable DDL; constraint violations correctly rejected).

Table	Rows on RDS
site	1
space_usage	21
building	124
utility_type	3
meter	308
daily_reading	28,336
weather_daily	92
eia_price_monthly	3
tariff_rate	9
TOTAL	28,897

4c. Verification evidence



Screenshot 4.1 — RDS console: DB identifier a2-campus-energy, MySQL Community 8.4.11, status Available (account ID redacted).



Screenshot 4.2 — MySQL Workbench connected to the live RDS endpoint (hidden): 9 tables and SELECT COUNT(*) per table, total 28,897.

Part 5 — Business Queries

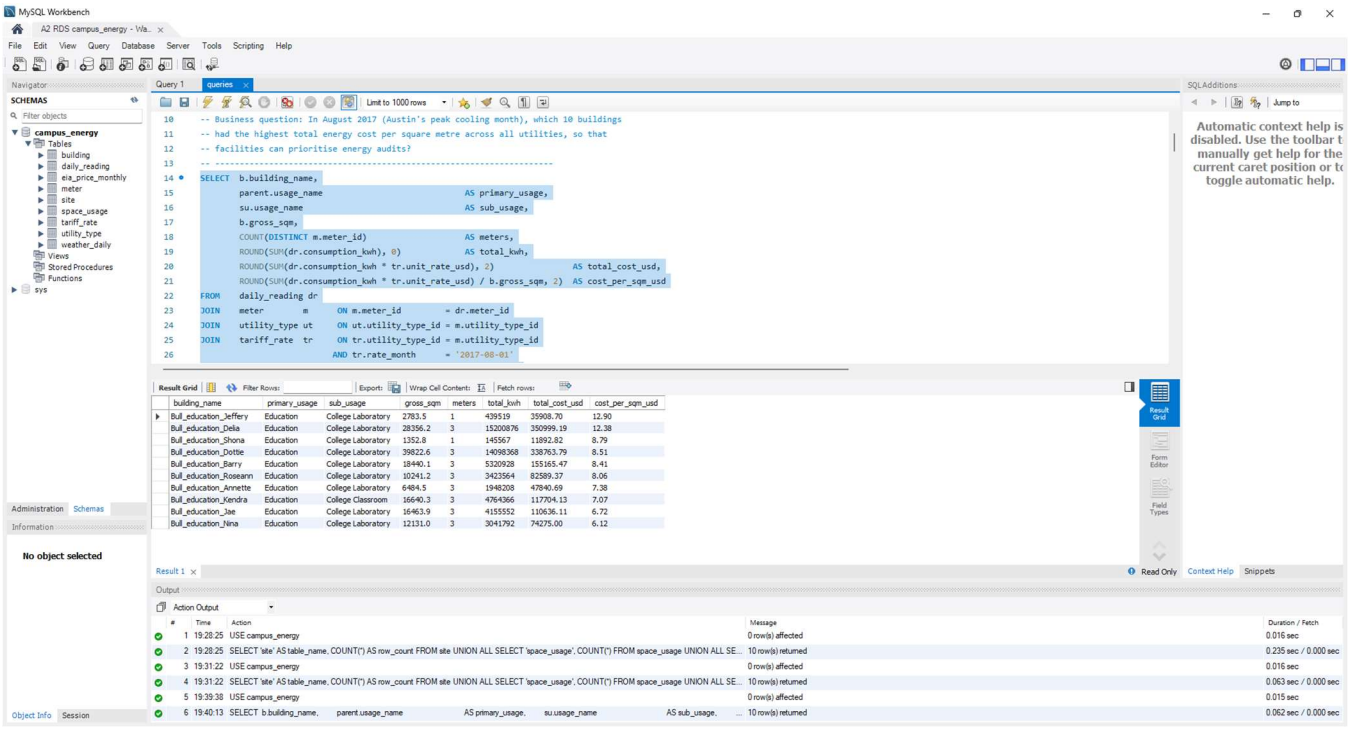
All 8 queries were executed on the live RDS instance. Coverage: multi-table INNER JOINs (Q1, Q3–Q6), LEFT OUTER (Q2, Q8), RIGHT OUTER (Q8), GROUP BY/HAVING (Q1, Q2, Q6), CTEs (Q3, Q4, Q7), window functions LAG (Q3), ROW_NUMBER + DENSE_RANK (Q4), AVG/STDDEV OVER (Q7), and a hierarchy self-join WITH ROLLUP (Q5).

Q1. INNER JOIN x6, GROUP BY, HAVING

Business question: In August 2017 (Austin's peak cooling month), which 10 buildings had the highest total energy cost per square metre across all utilities, so that facilities can prioritise energy audits?

```
SELECT  b.building_name,
        parent.usage_name                           AS primary_usage,
        su.usage_name                               AS sub_usage,
        b.gross_sqm,
        COUNT(DISTINCT m.meter_id)                  AS meters,
        ROUND(SUM(dr.consumption_kwh), 0)           AS total_kwh,
        ROUND(SUM(dr.consumption_kwh * tr.unit_rate_usd), 2)               AS total_cost_usd,
        ROUND(SUM(dr.consumption_kwh * tr.unit_rate_usd) / b.gross_sqm, 2)  AS cost_per_sqm_usd
FROM    daily_reading dr
JOIN    meter        m      ON m.meter_id         = dr.meter_id
JOIN    utility_type ut     ON ut.utility_type_id = m.utility_type_id
JOIN    tariff_rate  tr     ON tr.utility_type_id = m.utility_type_id
                           AND tr.rate_month      = '2017-08-01'
JOIN    building     b      ON b.building_id      = m.building_id
JOIN    space_usage  su     ON su.space_usage_id  = b.space_usage_id
JOIN    space_usage  parent ON parent.space_usage_id = su.parent_usage_id
WHERE   dr.reading_date >= '2017-08-01' AND dr.reading_date < '2017-09-01'
GROUP BY b.building_id, b.building_name, parent.usage_name, su.usage_name, b.gross_sqm
HAVING  SUM(dr.consumption_kwh) > 0
ORDER BY cost_per_sqm_usd DESC
LIMIT 10;
```

Result: All top-10 buildings are Education laboratories/classrooms; Bull_education_Jeffery leads at $12.90/m² for August — the audit shortlist.



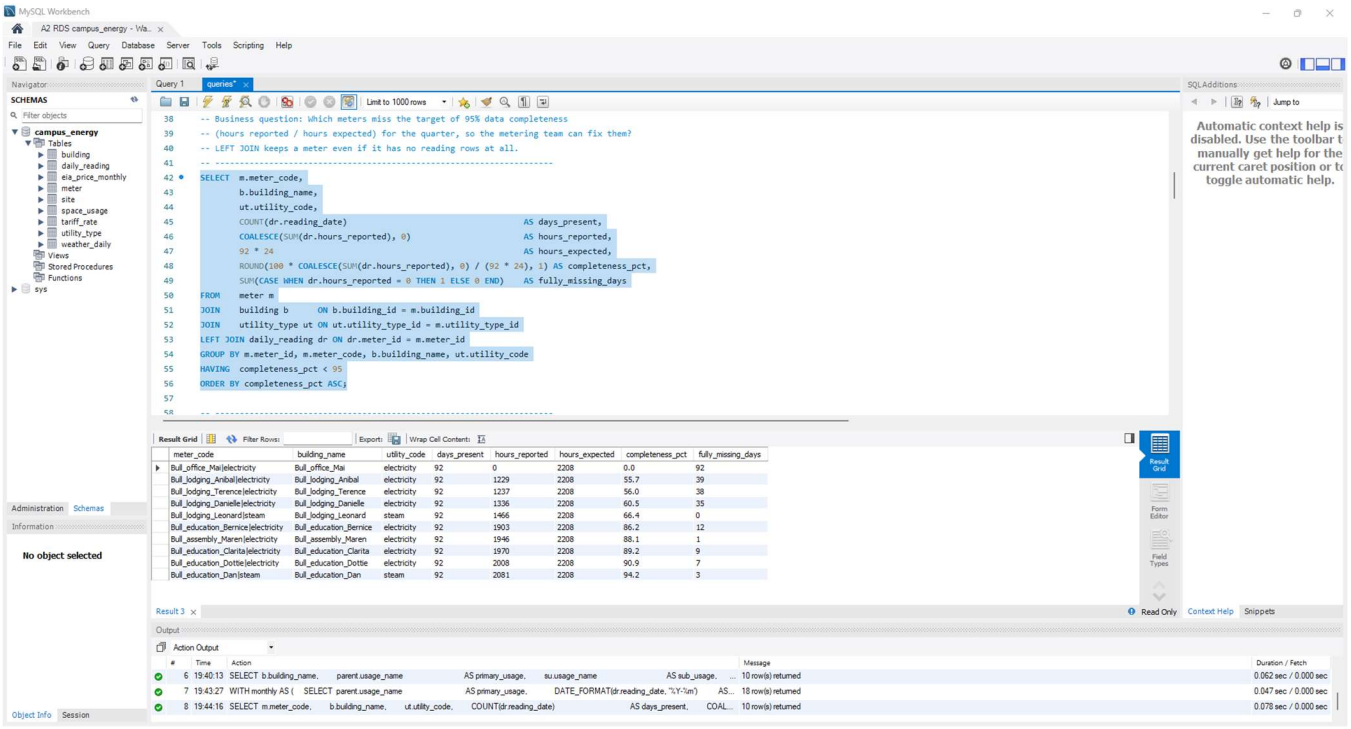
Screenshot 5.1 — Q1 executed on RDS with its result set.

Q2. LEFT OUTER JOIN, GROUP BY, HAVING

Business question: Which meters miss the A1 target of 95% data completeness (hours reported / hours expected) for the quarter, so the metering team can fix them? LEFT JOIN keeps a meter even if it has no reading rows at all.

```
SELECT  m.meter_code,
        b.building_name,
        ut.utility_code,
        COUNT(dr.reading_date)                                      AS days_present,
        COALESCE(SUM(dr.hours_reported), 0)                         AS hours_reported,
        92 * 24                                                     AS hours_expected,
        ROUND(100 * COALESCE(SUM(dr.hours_reported), 0) / (92 * 24), 1) AS completeness_pct,
        SUM(CASE WHEN dr.hours_reported = 0 THEN 1 ELSE 0 END)      AS fully_missing_days
FROM    meter m
JOIN    building b      ON b.building_id = m.building_id
JOIN    utility_type ut ON ut.utility_type_id = m.utility_type_id
LEFT JOIN daily_reading dr ON dr.meter_id = m.meter_id
GROUP BY m.meter_id, m.meter_code, b.building_name, ut.utility_code
HAVING  completeness_pct < 95
ORDER BY completeness_pct ASC;
```

Result: 10 meters miss the 95% target; Bull_office_Mai electricity reported 0 of 2,208 hours and three dormitory electricity meters are near 56–61%.



Screenshot 5.2 — Q2 executed on RDS with its result set.

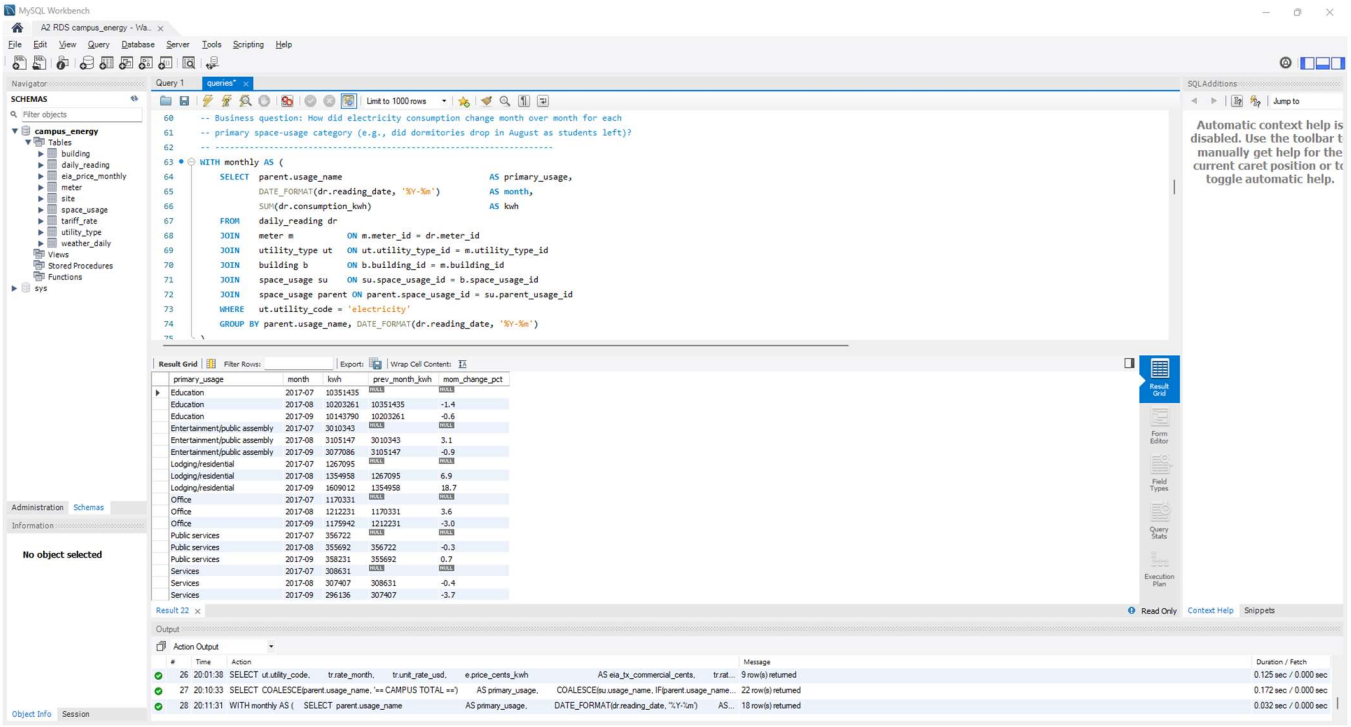
Q3. CTE + window function LAG

Business question: How did electricity consumption change month over month for each primary space-usage category (e.g., did dormitories drop in August as students left)?

```
WITH monthly AS (
    SELECT  parent.usage_name                               AS primary_usage,
            DATE_FORMAT(dr.reading_date, '%Y-%m')           AS month,
            SUM(dr.consumption_kwh)                         AS kwh
    FROM    daily_reading dr
    JOIN    meter m            ON m.meter_id = dr.meter_id
    JOIN    utility_type ut    ON ut.utility_type_id = m.utility_type_id
    JOIN    building b         ON b.building_id = m.building_id
    JOIN    space_usage su     ON su.space_usage_id = b.space_usage_id
    JOIN    space_usage parent ON parent.space_usage_id = su.parent_usage_id
    WHERE   ut.utility_code = 'electricity'
    GROUP BY parent.usage_name, DATE_FORMAT(dr.reading_date, '%Y-%m')
)
```

```
SELECT  primary_usage,
        month,
        ROUND(kwh, 0)                                                    AS kwh,
        ROUND(LAG(kwh) OVER (PARTITION BY primary_usage ORDER BY month), 0) AS prev_month_kwh,
        ROUND(100 * (kwh - LAG(kwh) OVER (PARTITION BY primary_usage ORDER BY month))
                  / LAG(kwh) OVER (PARTITION BY primary_usage ORDER BY month), 1) AS mom_change_pct
FROM    monthly
ORDER BY primary_usage, month;
```

Result: Dormitory electricity rose 6.9% in August and 18.7% in September (students returning), while Education fell slightly.



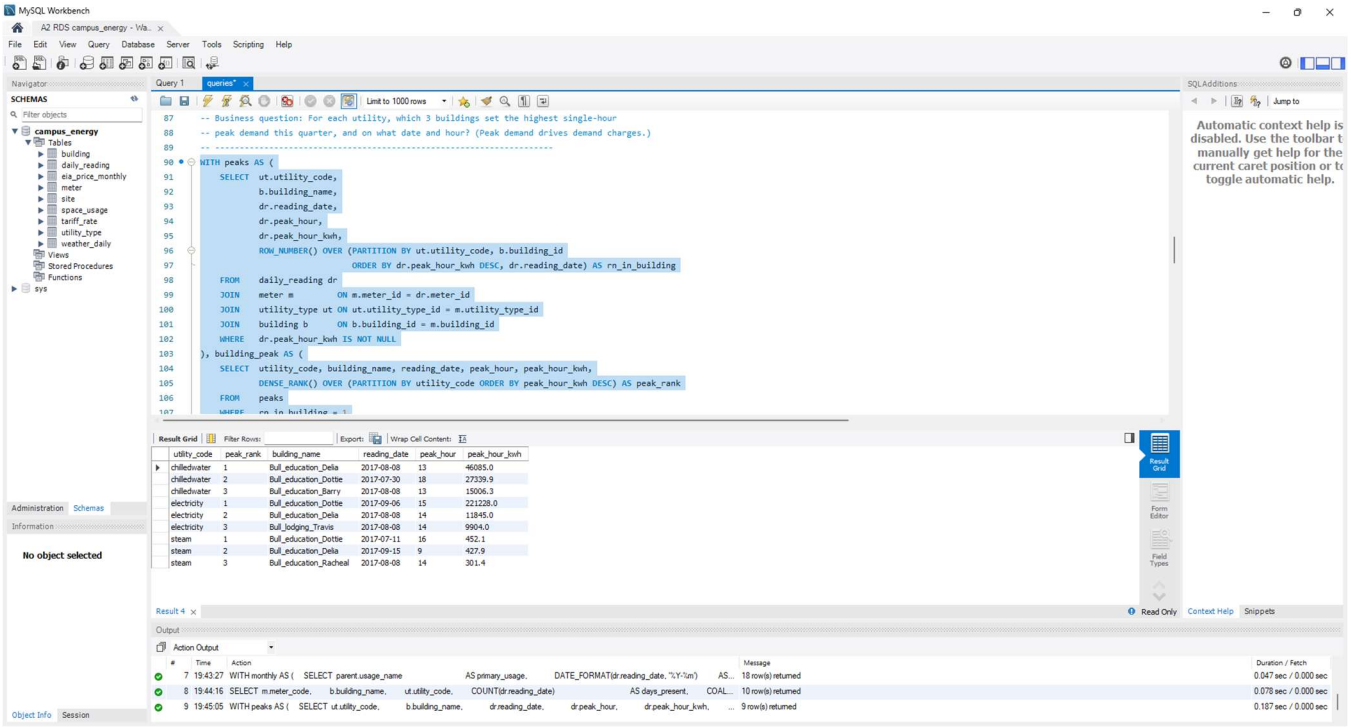
Screenshot 5.3 — Q3 executed on RDS with its result set.

Q4. CTE + window function ROW_NUMBER, multi-table INNER JOIN

Business question: For each utility, which 3 buildings set the highest single-hour peak demand this quarter, and on what date and hour? (Peak demand drives demand charges.)

```
WITH peaks AS (
    SELECT  ut.utility_code,
            b.building_name,
            dr.reading_date,
            dr.peak_hour,
            dr.peak_hour_kwh,
            ROW_NUMBER() OVER (PARTITION BY ut.utility_code, b.building_id
                               ORDER BY dr.peak_hour_kwh DESC, dr.reading_date) AS rn_in_building
    FROM    daily_reading dr
    JOIN    meter m          ON m.meter_id = dr.meter_id
    JOIN    utility_type ut ON ut.utility_type_id = m.utility_type_id
    JOIN    building b       ON b.building_id = m.building_id
    WHERE   dr.peak_hour_kwh IS NOT NULL
), building_peak AS (
    SELECT  utility_code, building_name, reading_date, peak_hour, peak_hour_kwh,
            DENSE_RANK() OVER (PARTITION BY utility_code ORDER BY peak_hour_kwh DESC) AS peak_rank
    FROM    peaks
    WHERE   rn_in_building = 1
)
SELECT  utility_code, peak_rank, building_name, reading_date, peak_hour,
        ROUND(peak_hour_kwh, 1) AS peak_hour_kwh
FROM    building_peak
WHERE   peak_rank <= 3
ORDER BY utility_code, peak_rank;
```

Result: Bull_education_Dottie's 221,228 kWh single-hour electricity peak (next highest 11,845) is almost certainly a meter fault; real chilled-water peaks cluster on 2017-08-08.



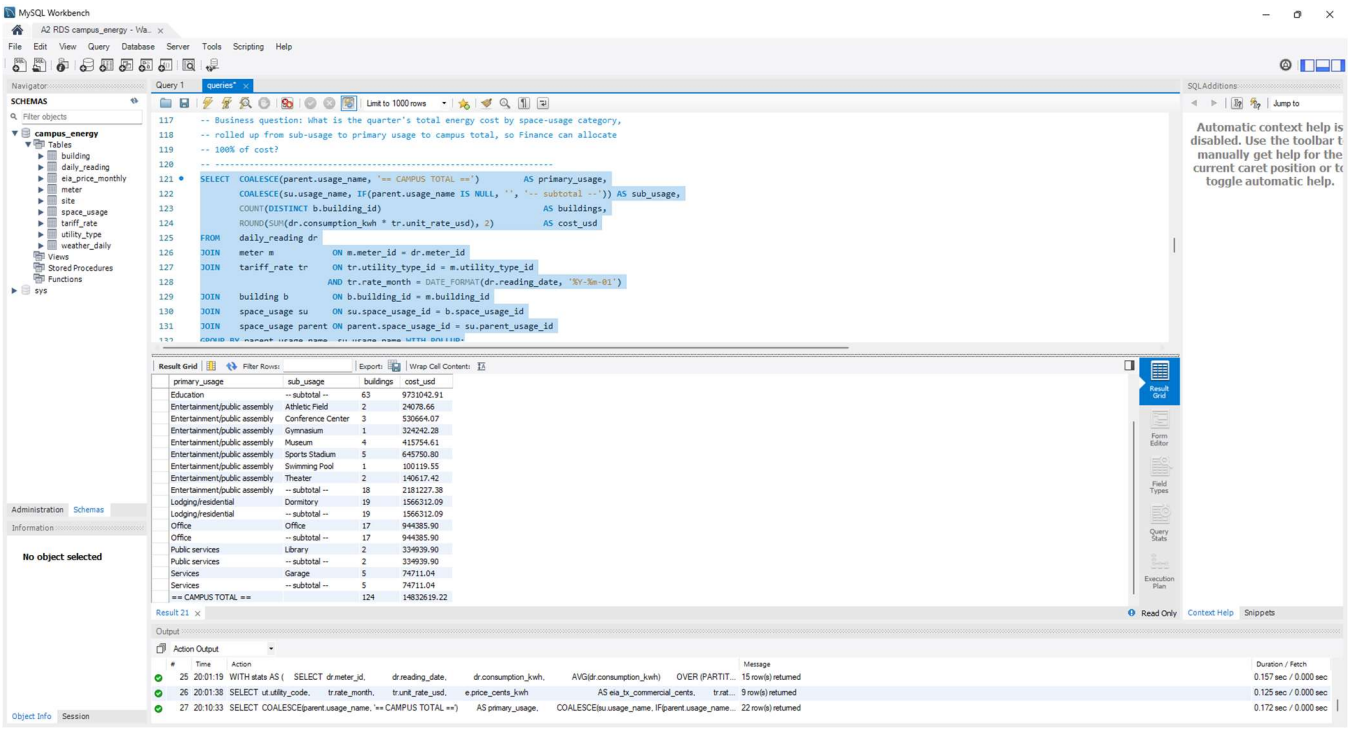
Screenshot 5.4 — Q4 executed on RDS with its result set.

Q5. Recursive hierarchy (self-join on space_usage), GROUP BY WITH ROLLUP

Business question: What is the quarter's total energy cost by space-usage category, rolled up from sub-usage to primary usage to campus total, so Finance can allocate 100% of cost (A1 success criterion)?

```
SELECT  COALESCE(parent.usage_name, '== CAMPUS TOTAL ==')         AS primary_usage,
        COALESCE(su.usage_name, IF(parent.usage_name IS NULL, '', '-- subtotal --')) AS sub_usage,
        COUNT(DISTINCT b.building_id)                               AS buildings,
        ROUND(SUM(dr.consumption_kwh * tr.unit_rate_usd), 2)        AS cost_usd
FROM    daily_reading dr
JOIN    meter m             ON m.meter_id = dr.meter_id
JOIN    tariff_rate tr      ON tr.utility_type_id = m.utility_type_id
                           AND tr.rate_month = DATE_FORMAT(dr.reading_date, '%Y-%m-01')
JOIN    building b          ON b.building_id = m.building_id
JOIN    space_usage su      ON su.space_usage_id = b.space_usage_id
JOIN    space_usage parent ON parent.space_usage_id = su.parent_usage_id
GROUP BY parent.usage_name, su.usage_name WITH ROLLUP;
```

Result: Rolls 100% of the quarter's cost from sub-usage to campus total: $14,832,619.22; Education accounts for $9.73M.



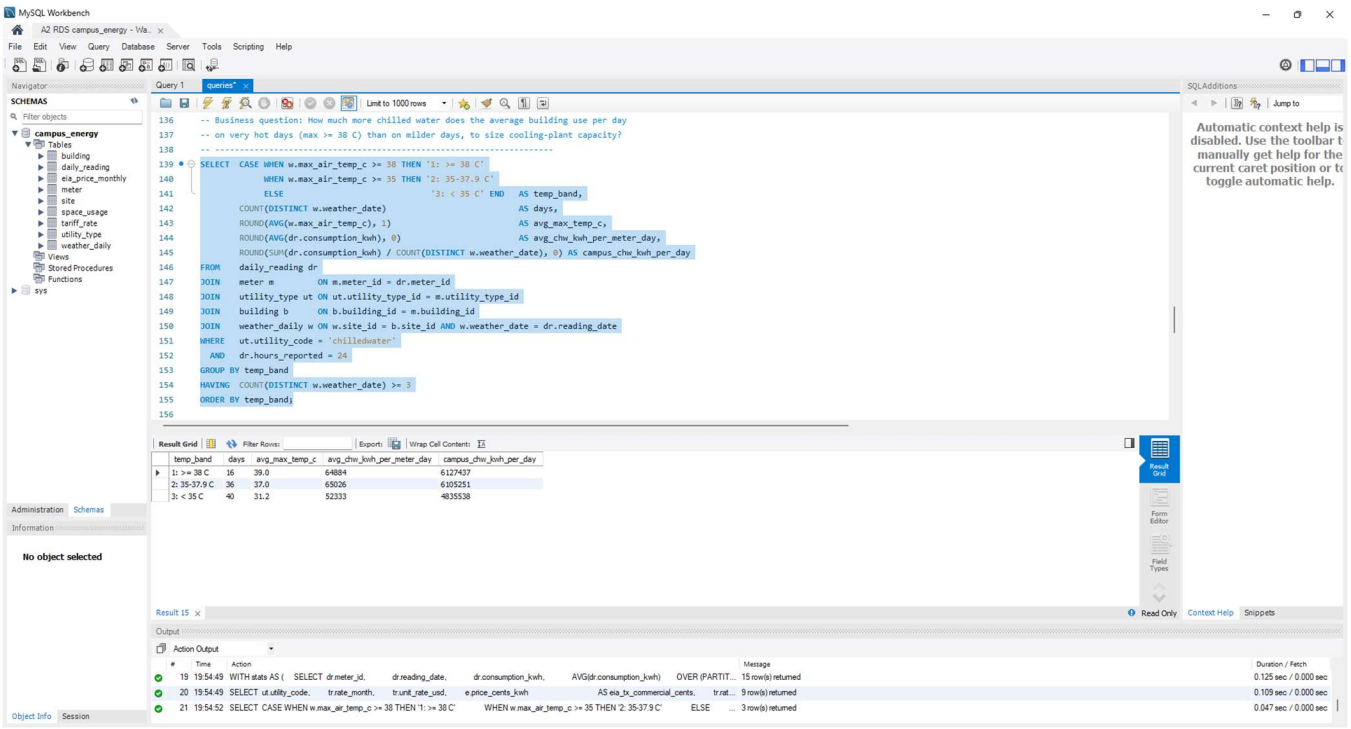
Screenshot 5.5 — Q5 executed on RDS with its result set.

Q6. INNER JOIN incl. weather, CASE banding, GROUP BY, HAVING

Business question: How much more chilled water does the average building use per day on very hot days (max >= 38 C) than on milder days, to size cooling-plant capacity?

```
SELECT  CASE WHEN w.max_air_temp_c >= 38 THEN '1: >= 38 C'
             WHEN w.max_air_temp_c >= 35 THEN '2: 35-37.9 C'
             ELSE                             '3: < 35 C' END   AS temp_band,
        COUNT(DISTINCT w.weather_date)                          AS days,
        ROUND(AVG(w.max_air_temp_c), 1)                         AS avg_max_temp_c,
        ROUND(AVG(dr.consumption_kwh), 0)                       AS avg_chw_kwh_per_meter_day,
        ROUND(SUM(dr.consumption_kwh) / COUNT(DISTINCT w.weather_date), 0) AS campus_chw_kwh_per_day
FROM    daily_reading dr
JOIN    meter m          ON m.meter_id = dr.meter_id
JOIN    utility_type ut ON ut.utility_type_id = m.utility_type_id
JOIN    building b       ON b.building_id = m.building_id
JOIN    weather_daily w ON w.site_id = b.site_id AND w.weather_date = dr.reading_date
WHERE   ut.utility_code = 'chilledwater'
  AND   dr.hours_reported = 24
GROUP BY temp_band
HAVING  COUNT(DISTINCT w.weather_date) >= 3
ORDER BY temp_band;
```

Result: Campus chilled water averaged 6.13M kWh/day on ≥ 38 °C days vs 4.84M on < 35 °C days (+27%).



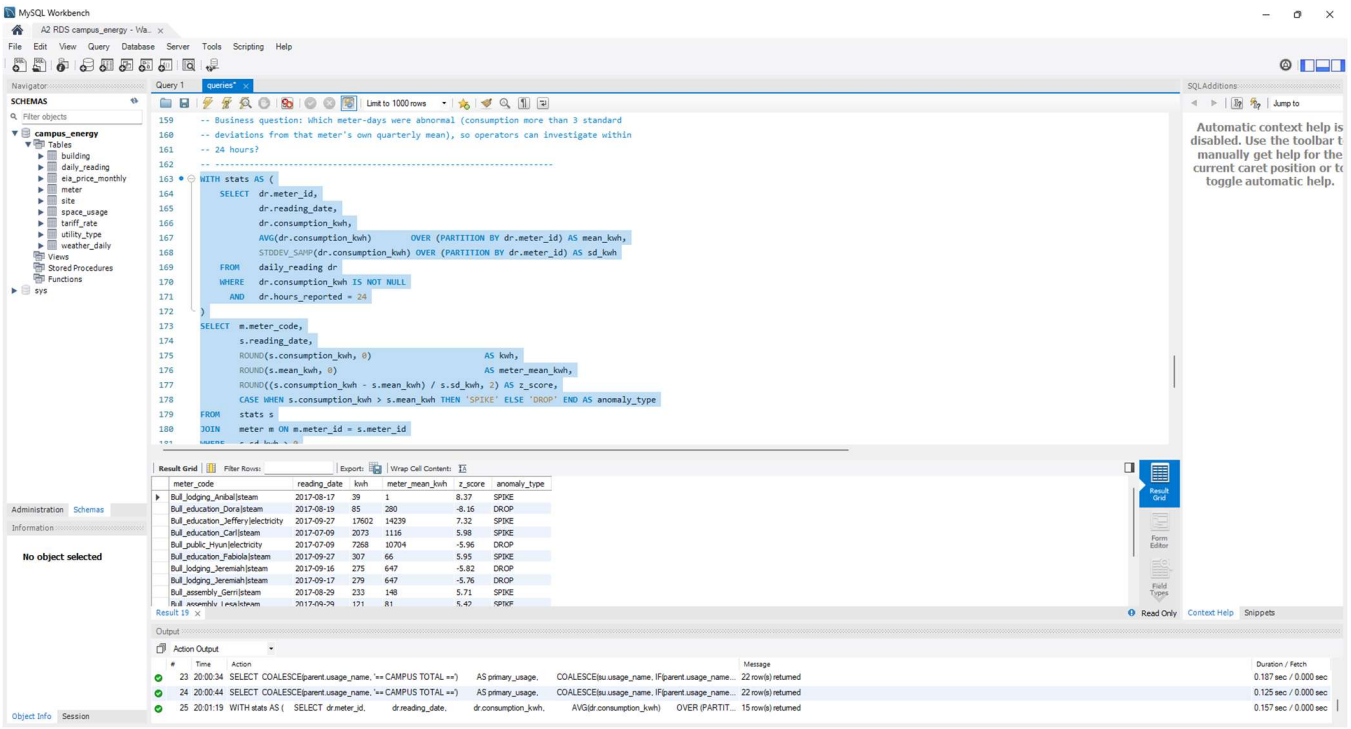
Screenshot 5.6 — Q6 executed on RDS with its result set.

Q7. CTE + window functions AVG/STDDEV OVER (PARTITION BY)

Business question: Which meter-days were abnormal (consumption more than 3 standard deviations from that meter's own quarterly mean), so operators can investigate within 24 hours (A1 success criterion)?

```
WITH stats AS (
    SELECT  dr.meter_id,
            dr.reading_date,
            dr.consumption_kwh,
            AVG(dr.consumption_kwh)        OVER (PARTITION BY dr.meter_id) AS mean_kwh,
            STDDEV_SAMP(dr.consumption_kwh) OVER (PARTITION BY dr.meter_id) AS sd_kwh
    FROM    daily_reading dr
    WHERE   dr.consumption_kwh IS NOT NULL
      AND   dr.hours_reported = 24
)
SELECT  m.meter_code,
        s.reading_date,
        ROUND(s.consumption_kwh, 0)                       AS kwh,
        ROUND(s.mean_kwh, 0)                              AS meter_mean_kwh,
        ROUND((s.consumption_kwh - s.mean_kwh) / s.sd_kwh, 2) AS z_score,
        CASE WHEN s.consumption_kwh > s.mean_kwh THEN 'SPIKE' ELSE 'DROP' END AS anomaly_type
FROM    stats s
JOIN    meter m ON m.meter_id = s.meter_id
WHERE   s.sd_kwh > 0
  AND   ABS(s.consumption_kwh - s.mean_kwh) > 3 * s.sd_kwh
ORDER BY ABS((s.consumption_kwh - s.mean_kwh) / s.sd_kwh) DESC
LIMIT 15;
```

Result: 15 meter-days exceed 3σ from their meter's mean (e.g., Bull_lodging_Anibal steam z = 8.37) — candidates for the 24-hour anomaly review.



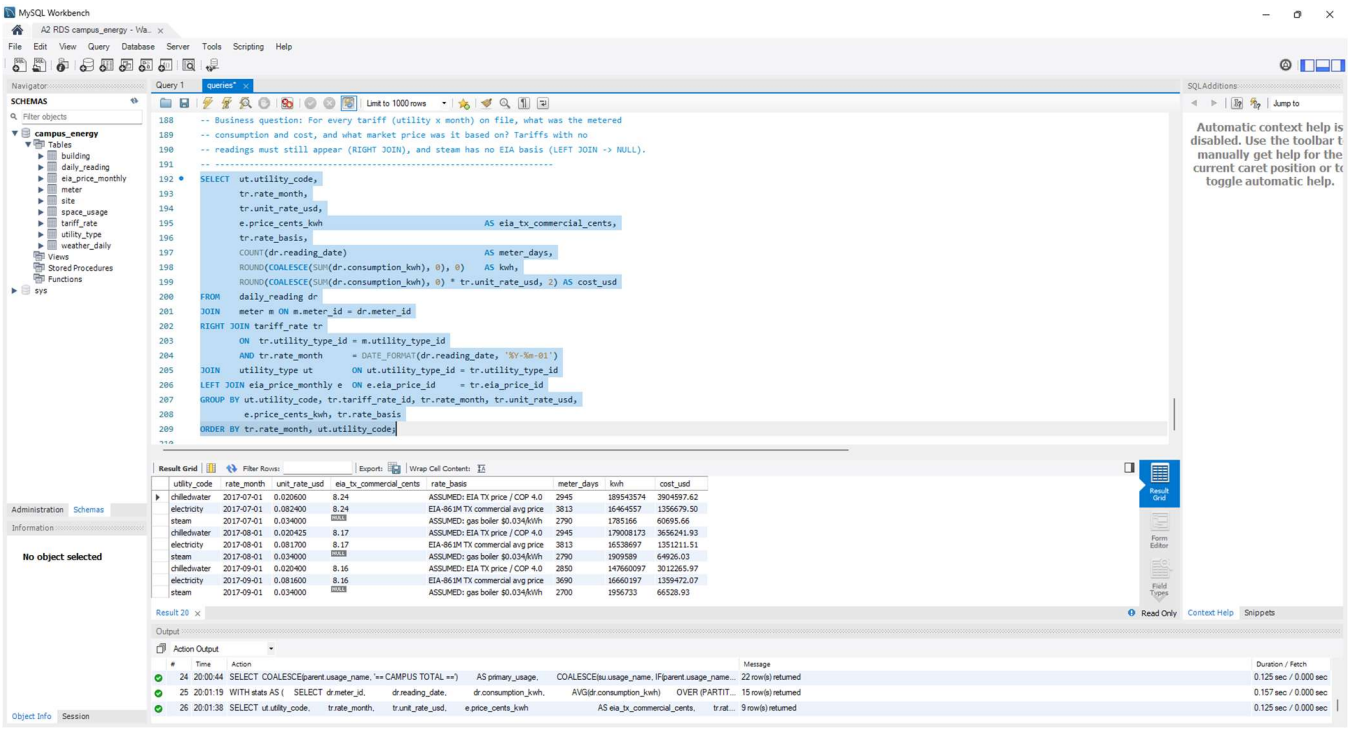
Screenshot 5.7 — Q7 executed on RDS with its result set.

Q8. RIGHT OUTER JOIN + LEFT OUTER JOIN, GROUP BY

Business question: For every tariff (utility x month) on file, what was the metered consumption and cost, and what market price was it based on? Tariffs with no readings must still appear (RIGHT JOIN), and steam has no EIA basis (LEFT JOIN -> NULL).

```
SELECT  ut.utility_code,
        tr.rate_month,
        tr.unit_rate_usd,
        e.price_cents_kwh                                   AS eia_tx_commercial_cents,
        tr.rate_basis,
        COUNT(dr.reading_date)                              AS meter_days,
        ROUND(COALESCE(SUM(dr.consumption_kwh), 0), 0)      AS kwh,
        ROUND(COALESCE(SUM(dr.consumption_kwh), 0) * tr.unit_rate_usd, 2) AS cost_usd
FROM    daily_reading dr
JOIN    meter m ON m.meter_id = dr.meter_id
RIGHT JOIN tariff_rate tr
        ON  tr.utility_type_id = m.utility_type_id
        AND tr.rate_month      = DATE_FORMAT(dr.reading_date, '%Y-%m-01')
JOIN    utility_type ut        ON ut.utility_type_id = tr.utility_type_id
LEFT JOIN eia_price_monthly e  ON e.eia_price_id     = tr.eia_price_id
GROUP BY ut.utility_code, tr.tariff_rate_id, tr.rate_month, tr.unit_rate_usd,
         e.price_cents_kwh, tr.rate_basis
ORDER BY tr.rate_month, ut.utility_code;
```

Result: Every tariff on file has readings; steam shows a NULL EIA basis (assumed rate) via the LEFT JOIN.



Screenshot 5.8 — Q8 executed on RDS with its result set.

Part 6 — Optimization, With Proof

Evidence: evidence/phase6_experiment.txt, captured on RDS MySQL 8.4.11 by an order-safe script (baseline, then three index hypotheses, each plan run 3 times; medians reported). A first before/after attempt was discarded because the index already existed when the baseline was taken.

6a. Execution plans — two most join-heavy queries

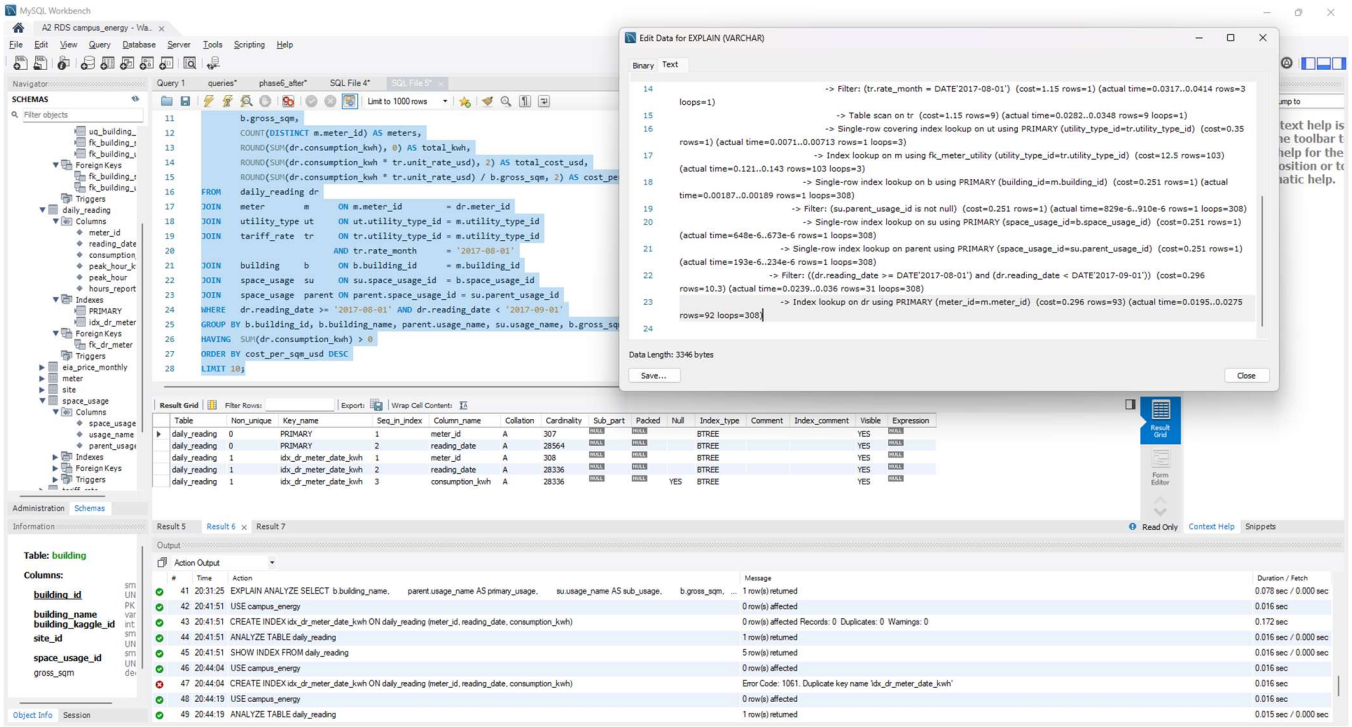
Q1 (7 tables) and **Q5** (6 tables). Baseline plans (full text in Appendix A):

- Q1: daily_reading read via 'Index lookup on dr using PRIMARY (meter_id=m.meter_id)', rows=92 × loops=308 = 28,336 examined, 9,548 kept after the date filter; median 44.3 ms.
- Q5: tariff_rate joined before daily_reading through the non-sargable DATE_FORMAT condition, so rows=92 × loops=924 = 85,008 examined to produce 28,336; median 124 ms.

6b. Index hypothesis and before/after

Hypothesis H1: **CREATE INDEX idx_dr_meter_date_kwh ON daily_reading (meter_id, reading_date, consumption_kwh);** — a covering index on Q1's per-meter lookup path. Two alternatives were also tested.

Run set (RDS 8.4.11)	Access on daily_reading	Rows examined → kept	Q1 ms (3 runs)	Median
Baseline (PK only)	PRIMARY index lookup	28,336 → 9,548	44.3 / 41.5 / 53.6	44.3
H1 (meter_id, reading_date, consumption_kwh)	unchanged — PRIMARY	28,336 → 9,548	42.3 / 42.4 / 52.0	42.4
H2 (reading_date, meter_id, consumption_kwh)	unchanged — PRIMARY	28,336 → 9,548	43.5 / 41.7 / 41.4	41.7
H3 tariff_rate (rate_month, utility_type_id, unit_rate_usd)	unchanged; join reordered (tr scan 9 rows → single-row lookup)	28,336 → 9,548	41.8 / 42.0 / 42.9	42.0



Screenshot 6.1 — After: idx_dr_meter_date_kwh exists (SHOW INDEX) but EXPLAIN ANALYZE still shows 'Index lookup on dr using PRIMARY … rows=92 loops=308'.

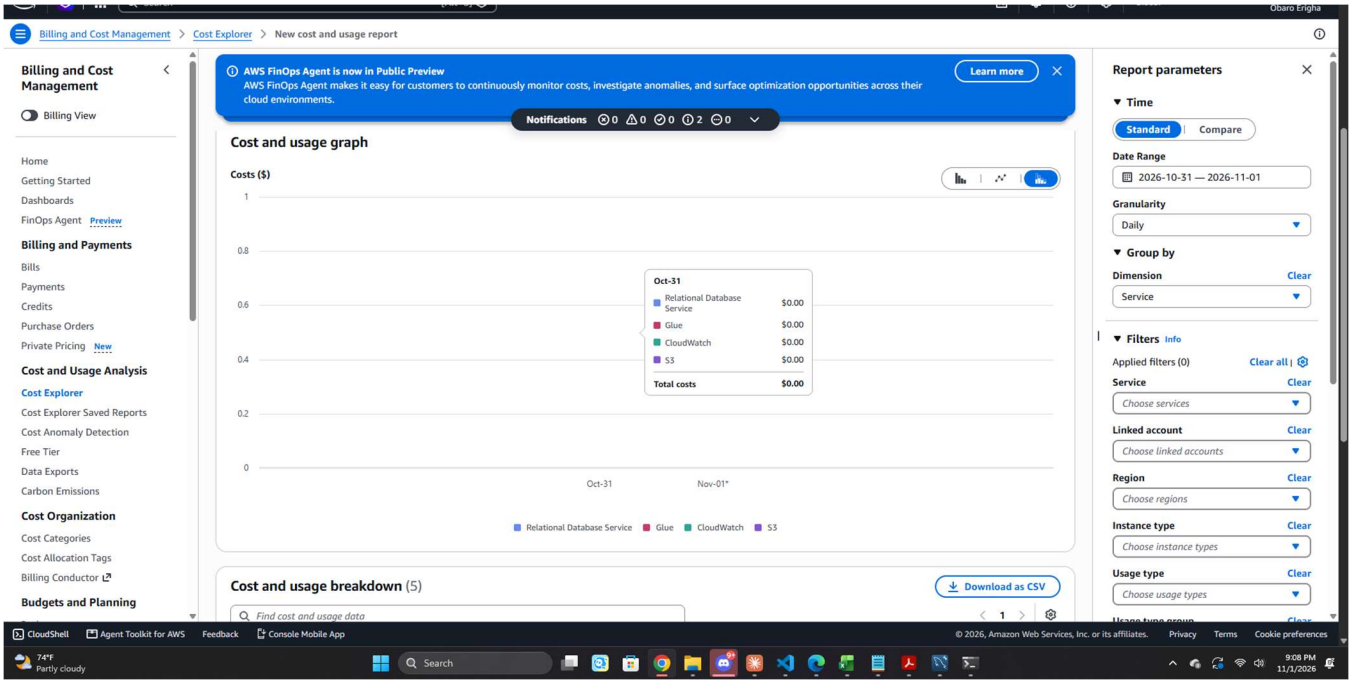
6c. Empirical analysis

On RDS MySQL 8.4.11, Q1's baseline plan reads daily_reading through the clustered primary key ('Index lookup on dr using PRIMARY (meter_id=m.meter_id)'), examining 92 rows × 308 loops = 28,336 rows and keeping only 31 per loop (9,548) after the date filter, for a median of 44.3 ms. The hypothesized covering index idx_dr_meter_date_kwh did not improve performance: the optimizer kept the PRIMARY lookup with the same 28,336 rows examined, and the 42.4 ms median sits inside the baseline's 41.5–53.6 ms run-to-run spread, because InnoDB's clustered key (meter_id, reading_date) already holds every column Q1 needs (secondary path costed 0.296 vs 0.297 per lookup) and a ref lookup can only use the equality prefix meter_id, so the August range is still applied as a filter. A date-leading index (H2, 41.7 ms) and a tariff_rate index (H3, 42.0 ms, which only reordered the 9-row tariff scan) left the daily_reading access unchanged as well. The plans show where the time actually goes — the join completes at 12.8 ms while sorting and grouping 9,548 rows runs from 30.2 to 41.3 ms — so the remaining cost is post-join work that no index on these tables removes; Q5's 85,008-row over-read is a query/schema issue (non-sargable DATE_FORMAT join), and no index was retained in production.

Part 7 — Cost, Teardown & Cloud Hygiene

7a. Billing

The instance ran from 18:56 to 21:15 CST on 2026-11-01 (≈2.3 hours; 2026-11-02 00:56–03:13 UTC). Estimated actual cost: db.t3.micro $0.017/h × 2.3 h ≈ $0.04, plus 20 GiB gp2 and public IPv4 ≈ $0.02, ≈ $0.06 total, covered by free-plan credits. A $1 monthly budget alarm was in place before any resource was created.



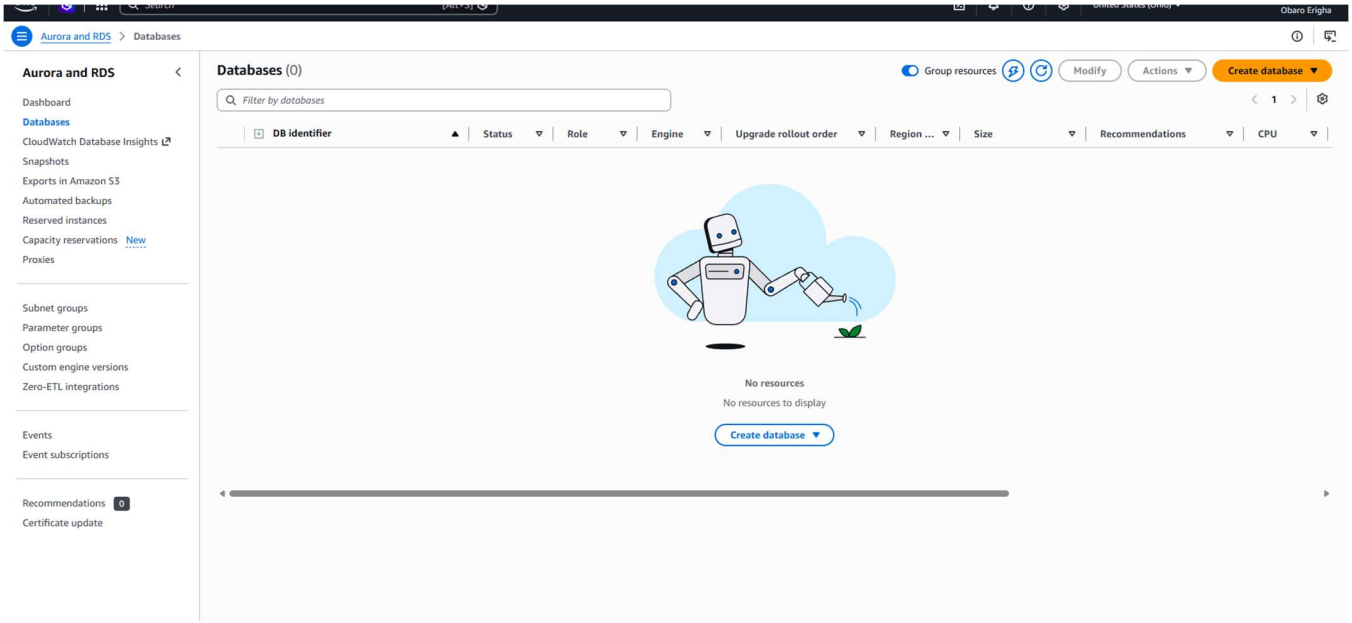
Screenshot 7.1 — Cost Explorer (daily, grouped by Service) listing Relational Database Service.

Note: AWS Cost Explorer data lags actual usage by up to 24 hours, and this account's usage is offset by free-plan credits; the screenshot was captured shortly before teardown, so the RDS line shows $0.00 rather than the ≈$0.06 incurred (which accrues to 2026-11-02 UTC).

7b. Teardown

```
aws rds delete-db-instance --db-instance-identifier a2-campus-energy --skip-final-snapshot --delete-automated-backups --
region us-east-2
aws rds wait db-instance-deleted --db-instance-identifier a2-campus-energy --region us-east-2      # returned 21:15:12 CST
aws ec2 delete-security-group --group-id <sg> --region us-east-2                                    # returned True
21:15:22 CST
```

Post-teardown sweep: 0 RDS instances, 0 automated backups, 0 Elastic IPs; only the default security group remains.



Screenshot 7.2 — RDS console, US East (Ohio): Databases (0) — instance deleted.

7c. Production 24/7 cost estimate

Component (us-east-2, MySQL)	Rate (AWS Pricing API)	Math	$/month
db.t3.micro Multi-AZ instance	$0.034/h	0.034 × 730 h	$24.82
20 GiB gp3 storage, Multi-AZ	$0.23/GB-mo	0.23 × 20	$4.60
7-day automated backups	$0.095/GB-mo beyond free	≤ 20 GB free (= provisioned storage)	$0.00
Public IPv4	$0.005/h	0 (private subnet in production)	$0.00
Total			≈ $29.42 (≈ $353/yr)

Operating this database 24/7 in production as a Multi-AZ db.t3.micro with 20 GiB gp3 storage and 7-day backups would cost about $29.42 per month — $24.82 for the instance (0.034 × 730 h) plus $4.60 for storage (0.23 × 20 GB), with backups inside the free allowance — or roughly $353 per year in us-east-2. Because the platform's load is steady and always-on, purchasing a 1-year No-Upfront Reserved Instance ($0.0239/h) lowers the instance charge to $17.45 per month (−30%), bringing the total to about $22 per month with no upfront payment, while keeping the instance private and on MySQL 8.4 avoids the public-IPv4 and 8.0 Extended Support surcharges.

Appendix A — Full EXPLAIN ANALYZE Output (RDS MySQL 8.4.11)

Q1 — baseline (run 2 of 3)

```
-> Limit: 10 row(s)  (actual time=41.5..41.5 rows=10 loops=1)
    -> Sort: cost_per_sqm_usd DESC  (actual time=41.5..41.5 rows=10 loops=1)
        -> Filter: (`sum(daily_reading.consumption_kwh)` > 0)  (actual time=30.4..41.4 rows=124 loops=1)
            -> Stream results  (actual time=30.4..41.4 rows=124 loops=1)
                -> Group aggregate: sum(daily_reading.consumption_kwh), sum(tmp_field), sum(tmp_field),
sum(daily_reading.consumption_kwh), count(distinct meter.meter_id)  (actual time=30.4..41.3 rows=124 loops=1)
                    -> Sort: b.building_id, b.building_name, parent.usage_name, su.usage_name, b.gross_sqm  (actual
time=30.2..31.5 rows=9548 loops=1)
                        -> Stream results  (cost=1122 rows=1078) (actual time=0.208..20.7 rows=9548 loops=1)
                            -> Nested loop inner join  (cost=1122 rows=1078) (actual time=0.202..12.8 rows=9548 loops=1)
                                -> Nested loop inner join  (cost=122 rows=103) (actual time=0.172..1.38 rows=308 loops=1)
                                    -> Nested loop inner join  (cost=85.9 rows=103) (actual time=0.17..1.27 rows=308 loops=1)
                                        -> Nested loop inner join  (cost=49.9 rows=103) (actual time=0.165..0.986 rows=308
loops=1)
                                            -> Nested loop inner join  (cost=14 rows=103) (actual time=0.16..0.499 rows=308
loops=1)
                                                -> Nested loop inner join  (cost=1.5 rows=1) (actual time=0.0362..0.0536 rows=3
loops=1)
                                                    -> Filter: (tr.rate_month = DATE'2017-08-01')  (cost=1.15 rows=1) (actual
time=0.0266..0.0347 rows=3 loops=1)
                                                        -> Table scan on tr  (cost=1.15 rows=9) (actual time=0.0237..0.0294 rows=9
loops=1)
                                                    -> Single-row covering index lookup on ut using PRIMARY
(utility_type_id=tr.utility_type_id)  (cost=0.35 rows=1) (actual time=0.00562..0.00576 rows=1 loops=3)
                                                -> Index lookup on m using fk_meter_utility (utility_type_id=tr.utility_type_id)
(cost=12.5 rows=103) (actual time=0.12..0.141 rows=103 loops=3)
                                            -> Single-row index lookup on b using PRIMARY (building_id=m.building_id)  (cost=0.251
rows=1) (actual time=0.00141..0.00143 rows=1 loops=308)
                                        -> Filter: (su.parent_usage_id is not null)  (cost=0.251 rows=1) (actual time=674e-
6..751e-6 rows=1 loops=308)
                                            -> Single-row index lookup on su using PRIMARY (space_usage_id=b.space_usage_id)
(cost=0.251 rows=1) (actual time=545e-6..570e-6 rows=1 loops=308)
                                    -> Single-row index lookup on parent using PRIMARY (space_usage_id=su.parent_usage_id)
(cost=0.251 rows=1) (actual time=180e-6..205e-6 rows=1 loops=308)
                                -> Filter: ((dr.reading_date >= DATE'2017-08-01') and (dr.reading_date < DATE'2017-09-01'))
(cost=0.297 rows=10.5) (actual time=0.0233..0.0351 rows=31 loops=308)
                                -> Index lookup on dr using PRIMARY (meter_id=m.meter_id)  (cost=0.297 rows=94.6) (actual
time=0.0191..0.0268 rows=92 loops=308)
```

Q1 — after index H1 idx_dr_meter_date_kwh (run 2 of 3)

```
-> Limit: 10 row(s)  (actual time=42.4..42.4 rows=10 loops=1)
    -> Sort: cost_per_sqm_usd DESC  (actual time=42.4..42.4 rows=10 loops=1)
        -> Filter: (`sum(daily_reading.consumption_kwh)` > 0)  (actual time=31..42.3 rows=124 loops=1)
            -> Stream results  (actual time=31..42.3 rows=124 loops=1)
                -> Group aggregate: sum(daily_reading.consumption_kwh), sum(tmp_field), sum(tmp_field),
sum(daily_reading.consumption_kwh), count(distinct meter.meter_id)  (actual time=31..42.2 rows=124 loops=1)
                    -> Sort: b.building_id, b.building_name, parent.usage_name, su.usage_name, b.gross_sqm  (actual
time=30.8..32.2 rows=9548 loops=1)
                        -> Stream results  (cost=1104 rows=1059) (actual time=0.202..20.8 rows=9548 loops=1)
                            -> Nested loop inner join  (cost=1104 rows=1059) (actual time=0.196..12.8 rows=9548 loops=1)
                                -> Nested loop inner join  (cost=122 rows=103) (actual time=0.165..1.42 rows=308 loops=1)
                                    -> Nested loop inner join  (cost=85.9 rows=103) (actual time=0.163..1.31 rows=308 loops=1)
                                        -> Nested loop inner join  (cost=49.9 rows=103) (actual time=0.159..1.02 rows=308 loops=1)
                                            -> Nested loop inner join  (cost=14 rows=103) (actual time=0.153..0.494 rows=308
loops=1)
                                                -> Nested loop inner join  (cost=1.5 rows=1) (actual time=0.0323..0.05 rows=3
loops=1)
                                                    -> Filter: (tr.rate_month = DATE'2017-08-01')  (cost=1.15 rows=1) (actual
time=0.0238..0.0318 rows=3 loops=1)
                                                        -> Table scan on tr  (cost=1.15 rows=9) (actual time=0.0209..0.0263 rows=9
loops=1)
                                                    -> Single-row covering index lookup on ut using PRIMARY
(utility_type_id=tr.utility_type_id)  (cost=0.35 rows=1) (actual time=0.0054..0.00544 rows=1 loops=3)
                                                -> Index lookup on m using fk_meter_utility (utility_type_id=tr.utility_type_id)
(cost=12.5 rows=103) (actual time=0.12..0.14 rows=103 loops=3)
                                            -> Single-row index lookup on b using PRIMARY (building_id=m.building_id)  (cost=0.251
rows=1) (actual time=0.00155..0.00157 rows=1 loops=308)
                                        -> Filter: (su.parent_usage_id is not null)  (cost=0.251 rows=1) (actual time=678e-
6..755e-6 rows=1 loops=308)
                                            -> Single-row index lookup on su using PRIMARY (space_usage_id=b.space_usage_id)
(cost=0.251 rows=1) (actual time=551e-6..576e-6 rows=1 loops=308)
                                    -> Single-row index lookup on parent using PRIMARY (space_usage_id=su.parent_usage_id)
(cost=0.251 rows=1) (actual time=190e-6..216e-6 rows=1 loops=308)
                                -> Filter: ((dr.reading_date >= DATE'2017-08-01') and (dr.reading_date < DATE'2017-09-01'))
(cost=0.296 rows=10.3) (actual time=0.0235..0.0351 rows=31 loops=308)
```

```
                                    -> Index lookup on dr using PRIMARY (meter_id=m.meter_id)  (cost=0.296 rows=92.8) (actual
time=0.0193..0.027 rows=92 loops=308)
```

Q5 — baseline (run 2 of 3)

```
-> Table scan on <temporary>  (cost=2.5..2.5 rows=0) (actual time=124..124 rows=22 loops=1)
    -> Temporary table  (cost=0..0 rows=0) (actual time=124..124 rows=22 loops=1)
        -> Group aggregate with rollup: sum(tmp_field), count(distinct building.building_id)  (actual time=94.5..124 rows=22
loops=1)
            -> Sort: usage_name, usage_name  (actual time=94.4..97.2 rows=28336 loops=1)
                -> Stream results  (cost=9409 rows=87365) (actual time=0.0865..71.3 rows=28336 loops=1)
                    -> Nested loop inner join  (cost=9409 rows=87365) (actual time=0.08..52.5 rows=28336 loops=1)
                        -> Nested loop inner join  (cost=408 rows=924) (actual time=0.0488..1.92 rows=924 loops=1)
                            -> Nested loop inner join  (cost=161 rows=308) (actual time=0.0389..0.547 rows=308 loops=1)
                                -> Nested loop inner join  (cost=99.5 rows=124) (actual time=0.0335..0.246 rows=124 loops=1)
                                    -> Nested loop inner join  (cost=56 rows=124) (actual time=0.0307..0.19 rows=124 loops=1)
                                        -> Covering index scan on b using fk_building_usage  (cost=12.7 rows=124) (actual
time=0.0199..0.0678 rows=124 loops=1)
                                        -> Filter: (su.parent_usage_id is not null)  (cost=0.251 rows=1) (actual time=707e-
6..804e-6 rows=1 loops=124)
                                            -> Single-row index lookup on su using PRIMARY (space_usage_id=b.space_usage_id)
(cost=0.251 rows=1) (actual time=548e-6..573e-6 rows=1 loops=124)
                                    -> Single-row index lookup on parent using PRIMARY (space_usage_id=su.parent_usage_id)
(cost=0.251 rows=1) (actual time=230e-6..271e-6 rows=1 loops=124)
                                -> Covering index lookup on m using uq_meter_building_utility (building_id=b.building_id)
(cost=0.252 rows=2.48) (actual time=0.00159..0.00219 rows=2.48 loops=124)
                            -> Index lookup on tr using uq_tariff_utility_month (utility_type_id=m.utility_type_id)  (cost=0.501
rows=3) (actual time=0.00358..0.0042 rows=3 loops=308)
                        -> Filter: (tr.rate_month = cast(date_format(dr.reading_date,'%Y-%m-01') as date))  (cost=0.297 rows=94.6)
(actual time=0.0297..0.053 rows=30.7 loops=924)
                            -> Index lookup on dr using PRIMARY (meter_id=m.meter_id)  (cost=0.297 rows=94.6) (actual
time=0.0185..0.0262 rows=92 loops=924)
```

Q5 — with index H1 (run 1)

```
-> Table scan on <temporary>  (cost=2.5..2.5 rows=0) (actual time=123..123 rows=22 loops=1)
    -> Temporary table  (cost=0..0 rows=0) (actual time=123..123 rows=22 loops=1)
        -> Group aggregate with rollup: sum(tmp_field), count(distinct building.building_id)  (actual time=95.2..123 rows=22
loops=1)
            -> Sort: usage_name, usage_name  (actual time=95..97.7 rows=28336 loops=1)
                -> Stream results  (cost=9250 rows=85779) (actual time=0.112..71.4 rows=28336 loops=1)
                    -> Nested loop inner join  (cost=9250 rows=85779) (actual time=0.105..52.5 rows=28336 loops=1)
                        -> Nested loop inner join  (cost=408 rows=924) (actual time=0.0707..1.96 rows=924 loops=1)
                            -> Nested loop inner join  (cost=161 rows=308) (actual time=0.0558..0.571 rows=308 loops=1)
                                -> Nested loop inner join  (cost=99.5 rows=124) (actual time=0.0482..0.272 rows=124 loops=1)
                                    -> Nested loop inner join  (cost=56 rows=124) (actual time=0.0442..0.212 rows=124 loops=1)
                                        -> Covering index scan on b using fk_building_usage  (cost=12.7 rows=124) (actual
time=0.0275..0.0826 rows=124 loops=1)
                                        -> Filter: (su.parent_usage_id is not null)  (cost=0.251 rows=1) (actual time=760e-
6..859e-6 rows=1 loops=124)
                                            -> Single-row index lookup on su using PRIMARY (space_usage_id=b.space_usage_id)
(cost=0.251 rows=1) (actual time=602e-6..629e-6 rows=1 loops=124)
                                    -> Single-row index lookup on parent using PRIMARY (space_usage_id=su.parent_usage_id)
(cost=0.251 rows=1) (actual time=249e-6..298e-6 rows=1 loops=124)
                                -> Covering index lookup on m using uq_meter_building_utility (building_id=b.building_id)
(cost=0.252 rows=2.48) (actual time=0.00155..0.00215 rows=2.48 loops=124)
                            -> Index lookup on tr using uq_tariff_utility_month (utility_type_id=m.utility_type_id)  (cost=0.501
rows=3) (actual time=0.00363..0.00424 rows=3 loops=308)
                        -> Filter: (tr.rate_month = cast(date_format(dr.reading_date,'%Y-%m-01') as date))  (cost=0.296 rows=92.8)
(actual time=0.0299..0.053 rows=30.7 loops=924)
                            -> Index lookup on dr using PRIMARY (meter_id=m.meter_id)  (cost=0.296 rows=92.8) (actual
time=0.0185..0.0261 rows=92 loops=924)
```

Appendix B — Notes, Reproducibility and Deliverables

Footnote: cost scale vs. Assignment 1

Assignment 1 valued campus electricity at about $5.4M per year. With real BDG2 consumption and real EIA prices, electricity alone totals $4,067,363 for the July–September 2017 quarter (Q8), and the full quarter across all utilities is $14,832,619.22 (chilled water dominates at ≈ 516 GWh-thermal priced at an assumed chiller COP of 4.0). Summer is Austin's peak season, so an annual figure would be lower than 4× the quarter, but the A1 estimate was clearly conservative; A1 is submitted and unchanged, and A2's figures supersede it.

Assumptions behind the cost figures

- Chilled water rate = TX commercial electricity price ÷ COP 4.0; steam = $0.034/kWh (≈ $8/MMBtu gas ÷ 0.8 boiler efficiency). Both are assumptions, flagged in the data (rate_basis column).

Deliverables

File	Purpose
A2_campus_energy.sql	DDL, load verification, 8 commented queries, Part 6 EXPLAIN + index commands
normalization_workbook.xlsx	Tabs 1NF / 2NF / 3NF with keys, FDs, real sample rows
scripts/build_flat_table.py	Option B: reproducible construction of the flat table from BDG2 + EIA
scripts/normalize.py, scripts/load_3nf.py	3NF decomposition (lossless check) and RDS loader
design/*.mmd, *.png	Editable ER diagram sources and renders
evidence/phase6_experiment.txt	Raw RDS execution plans

Peer review (optional bonus)

[Add peer review here if submitting for the +5 bonus.]